Agentic AI Engineering

Intelligent Multi-Source RAG and Query Orchestration Platform

Max Montaseri

- Designed a **multi-source RAG architecture** that dynamically routes natural-language questions between a Chroma vector knowledge base and relational accommodation database using structured LLM classification.
- Implemented **metadata-aware retrieval** with destination and region attributes, enabling semantic search combined with deterministic metadata filtering.
- Built structured LLM query generation using **Pydantic schemas and function calling**, converting natural-language questions into typed search objects and database-compatible filters.
- Implemented **SelfQueryRetriever** workflows to automatically infer metadata filters from user questions and improve retrieval precision.
- Developed a natural-language-to-SQL pipeline using LangChain's SQL utilities, including an LLM-based SQL normalization stage for executable query generation.
- Designed a **RAG Fusion pipeline** generating multiple semantic query perspectives and combining retrieval rankings using Reciprocal Rank Fusion.
- Implemented RRF scoring and ranking logic to merge results from multiple retrieval streams while selecting top-ranked evidence for downstream generation.
- Applied **LCEL** with RunnableLambda, RunnablePassthrough, mapped retrieval, structured outputs, and composable prompt/LLM/parser pipelines.

- Explored semantic SQL retrieval using embeddings and vector similarity to complement traditional exact-match relational queries.
- Established an extensible architecture for future adaptive retrieval, hybrid search, reranking, evaluation, guardrails, and LangGraph-based agentic orchestration.

Architecture

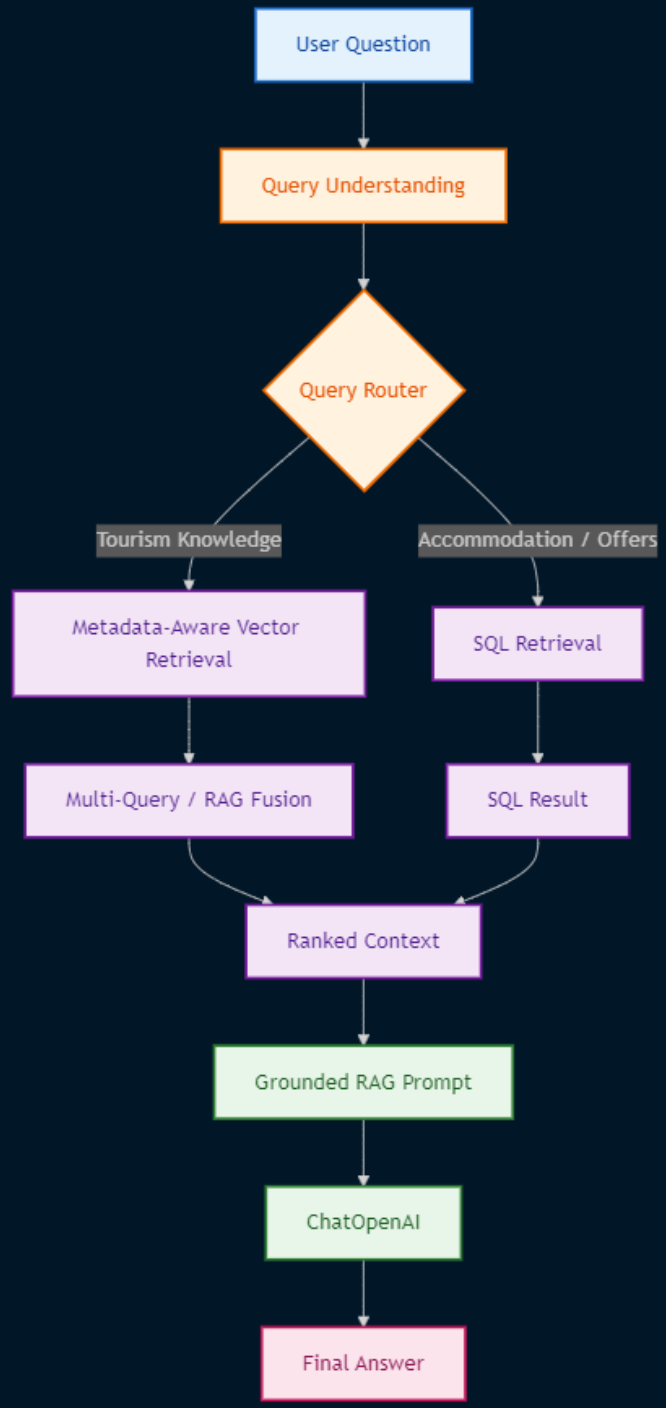
The project explores a central RAG engineering problem:

How should an AI application transform, route, filter, and rank a user's query before generating a grounded answer?

The source moves beyond basic vector similarity search and explores several retrieval dimensions:

1. Metadata filtering
2. LLM-generated metadata filters
3. Structured query generation
4. Relational database retrieval
5. Semantic database search
6. Retriever routing
7. Multi-query retrieval
8. Reciprocal Rank Fusion
9. RAG answer synthesis

The source implements separate vector and relational retrieval paths and uses an LLM-based structured router to choose between them



Metadata-Aware Retrieval

Why Metadata Matters

Pure vector similarity answers: "Which documents are semantically similar?"

Metadata filtering answers: "Which semantically relevant documents also satisfy explicit constraints?"

The source enriches documents with:

- source
- destination
- Region

before indexing them into Chroma.

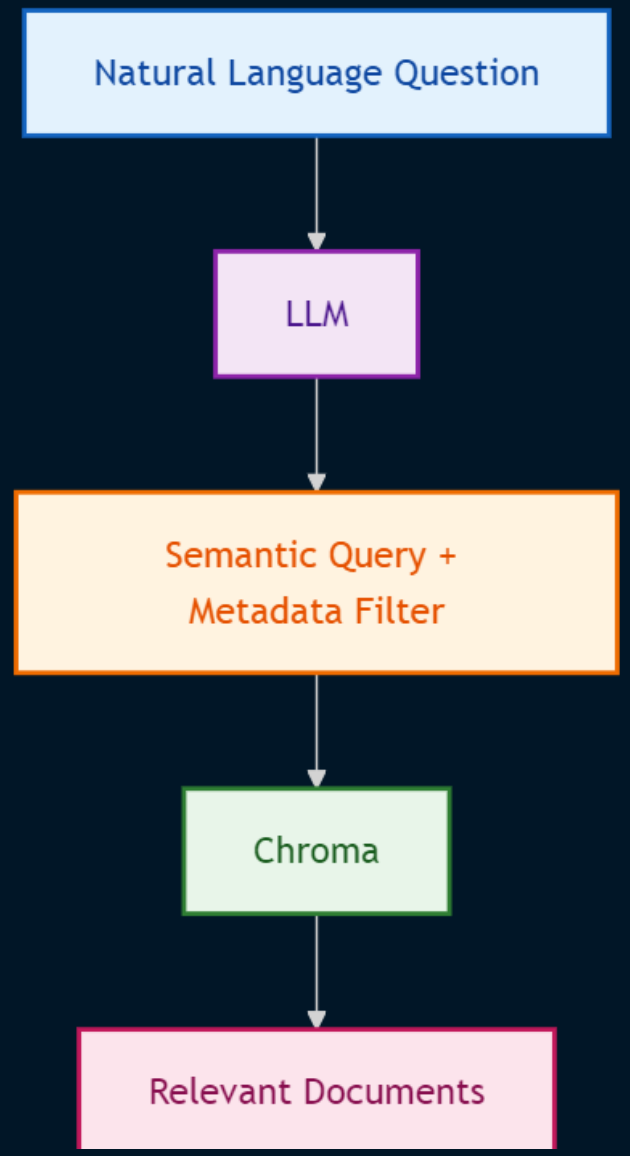
Ingestion Pipeline

Self-Query Retrieval

This approach introduces additional LLM cost and latency

Explicit Filtering
First explicitly applying a Chroma filter

SelfQueryRetriever
Then SelfQueryRetriever, allowing an LLM to infer metadata constraints from the natural-language question.



Structured Query Generation

Instead of allowing the retriever to internally infer the query, the application defines a strongly typed Pydantic schema

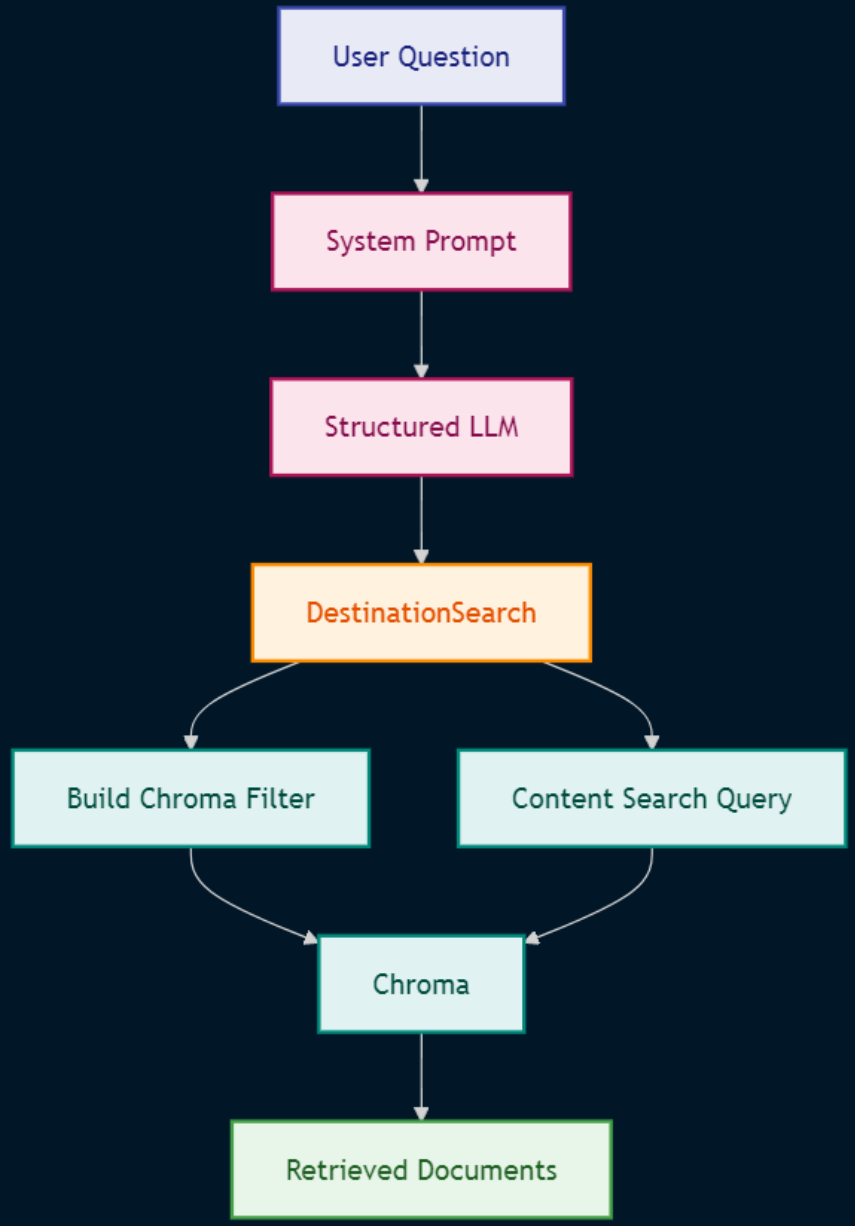
Produce the structured representation
llm.with_structured_output(DestinationSearch, method="function_calling")

Why This Pattern Is Valuable

It separates Question Understanding from Database Retrieval

This is a powerful enterprise AI pattern because retrieval infrastructure receives a predictable typed object instead of unrestricted natural-language instructions.

Structured LLM Flow



Natural Language to SQL

Relational retrieval

Create a SQL-generation chain and test it with accommodation-offer questions.

SQL Cleanup Pipeline

LLM output validation/normalization stage

Falure mode

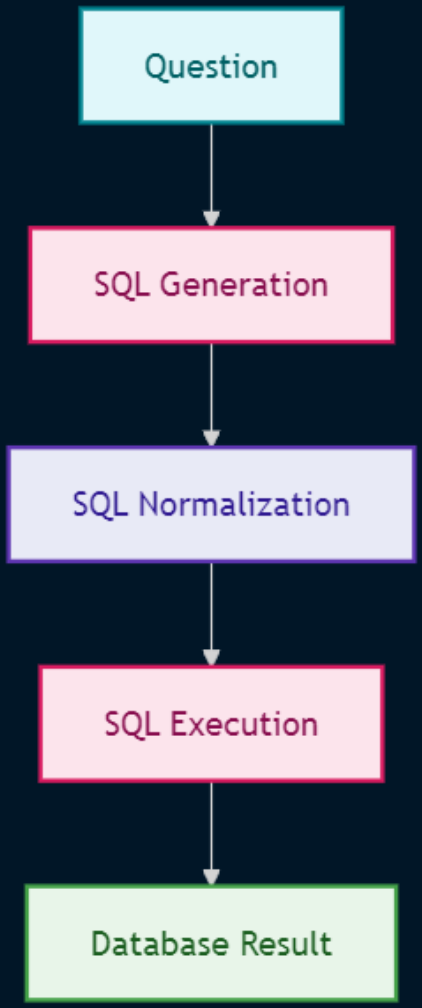
Generated SQL may contain Markdown code fences such as:

```sql
...
```

Adds a second LLM transformation to clean the generated SQL before execution.

The cleanup prompt requires:

- executable SQLite
- table-qualified columns
- terminating semicolon
- no Markdown
- no language labels



Semantic SQL Search

Go beyond conventional SQL and apply semantic SQL search using embeddings

Embedding values and using vector similarity to search by meaning rather than exact string matching

pgvector as an example of a PostgreSQL vector extension supporting similarity search

Traditional SQL

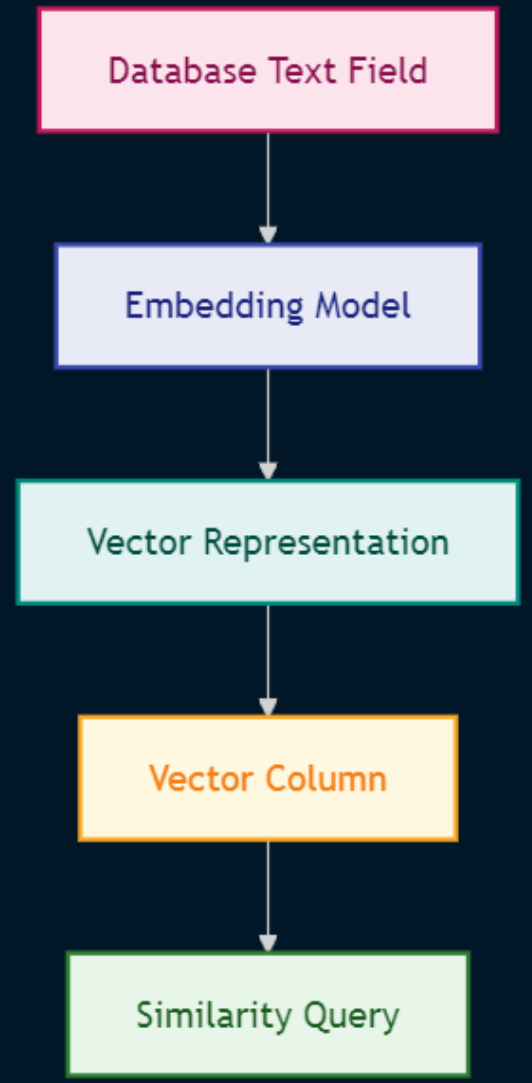
WHERE name = 'Roberto'

Semantic SQL

ORDER BY vector_similarity(query_embedding, stored_embedding)

Advantage
semantic retrieval can be combined with traditional filters, joins, and relational operations.

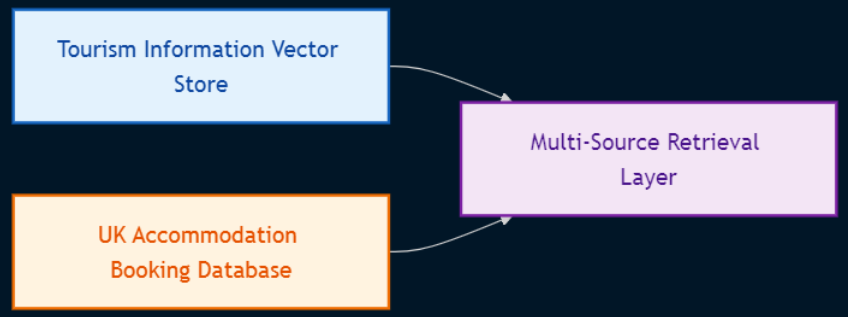
Also, combine semantic filtering with exact matching and multi-table queries



Query Routing

Retriever routing
One of the most important architectural patterns

The application has two conceptual data sources:



Pydantic router schema

RouteQuery with two possible destinations:
- tourism_info_store
- uk_booking_db

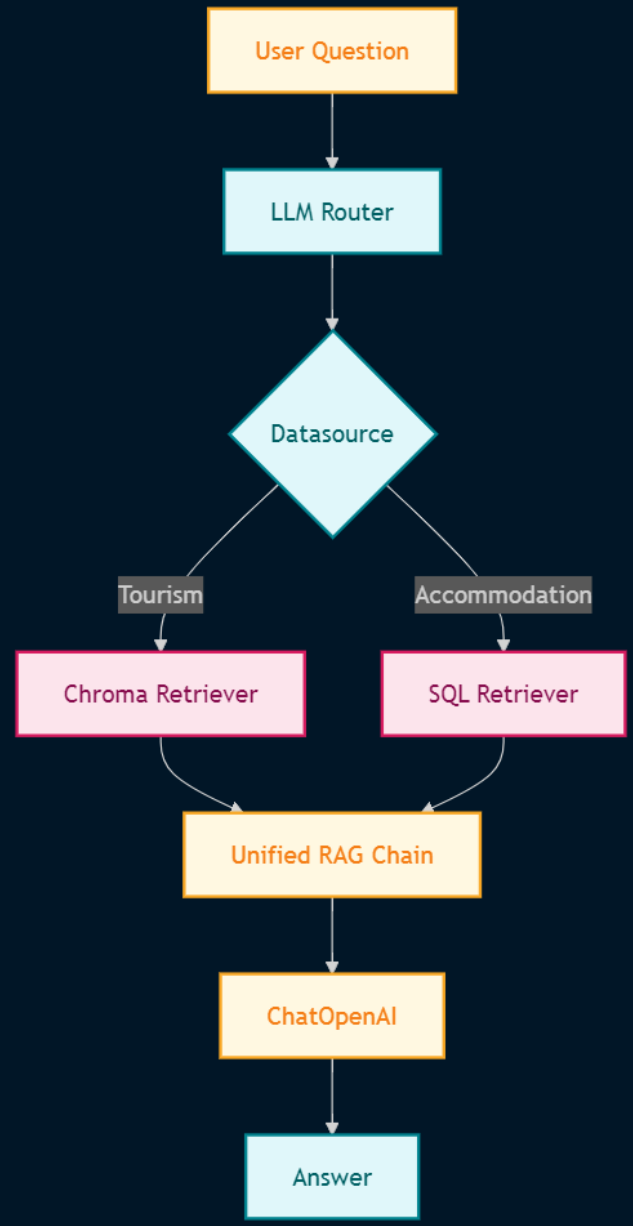
Why Routing Matters?

- This is an important step toward Agentic AI architecture
- Instead of forcing one retrieval technology to handle every question, the system selects the appropriate retrieval mechanism
- Router with structured LLM output and maps each routing value to a corresponding retriever chain

End-to-End Routed RAG

- A reusable function that constructs the final RAG chain from a selected retriever.
- The original question is preserved while the selected retriever supplies the context.
- The final prompt instructs the LLM to answer from the supplied context and handle structured accommodation results.

Router Architecture



RAG Fusion

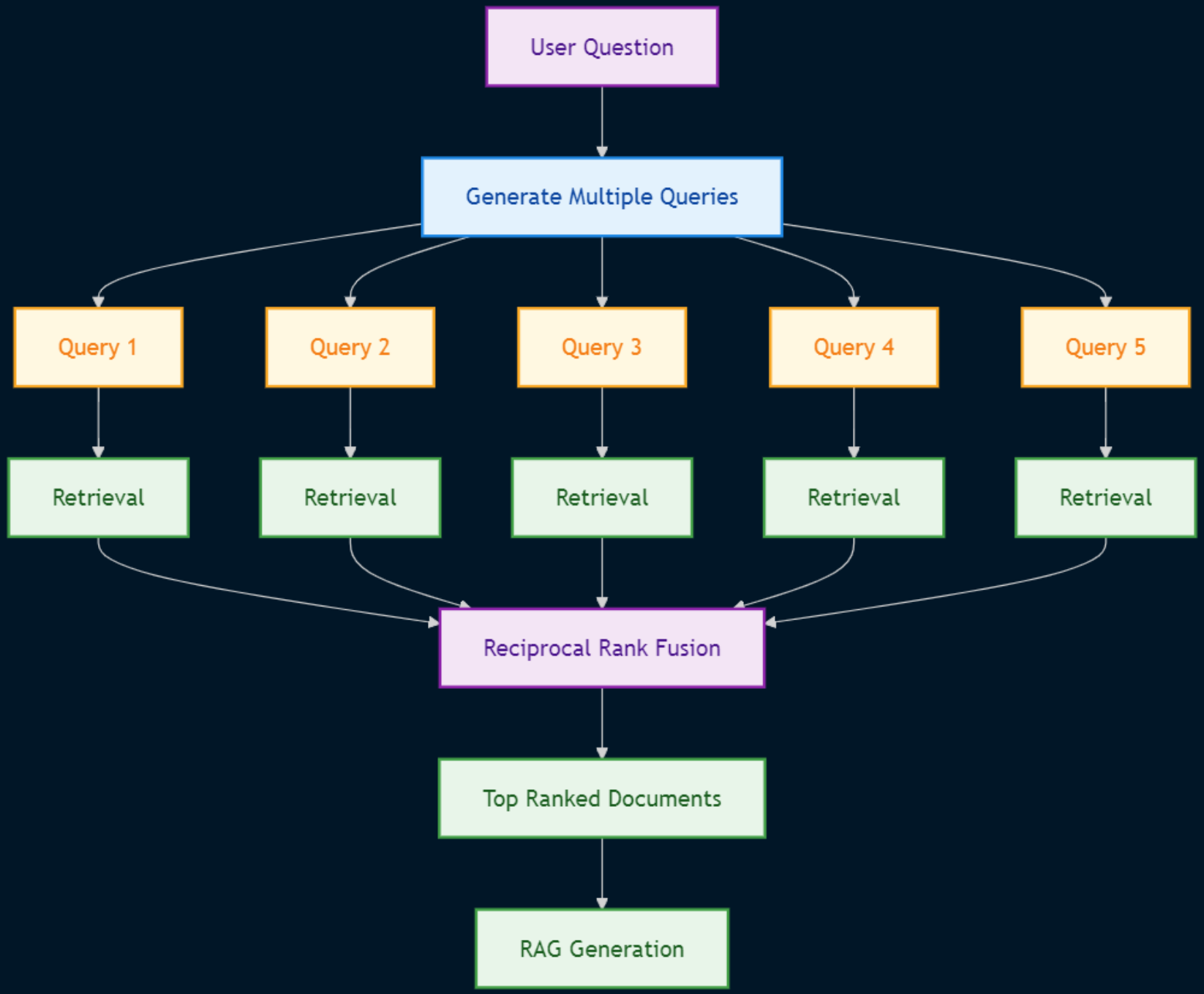
Problem

One query may not capture all relevant semantic perspectives.

Solution

Generate multiple queries, retrieve documents for each query, and combine the ranked result sets.

Generate five query variants using an LLM and then apply RRF to combine their retrieval results



Reciprocal Rank Fusion (RRF) Algorithm

RRF Score = Σ 1 / (rank + k)

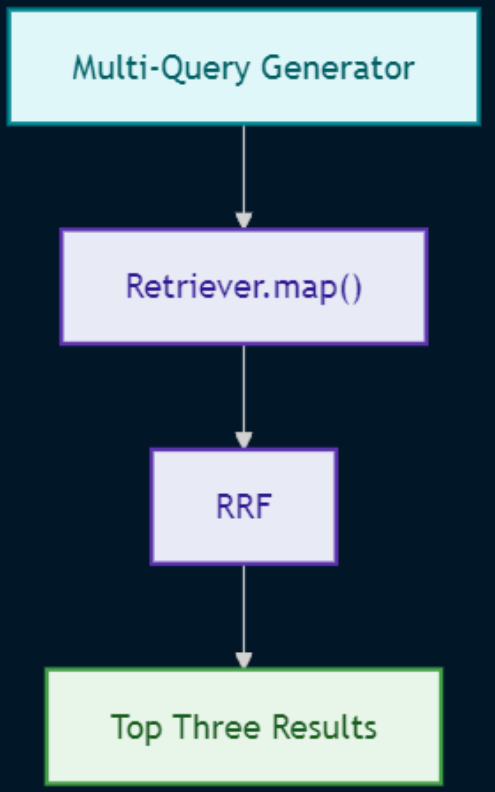
- rank is the document's position in a result list
- k is a smoothing constant
- multiple rankings contribute to the final score

Use k=60 by default and rerank documents based on accumulated scores.

Why RRF?

Different queries may retrieve different but relevant documents.
RRF provides a simple rank-based method for combining these retrieval signals without requiring the original retrieval scores to be directly comparable.

RAG Fusion Pipeline



The resulting retrieval chain is then inserted into a standard RAG prompt → LLM → output-parser pipeline

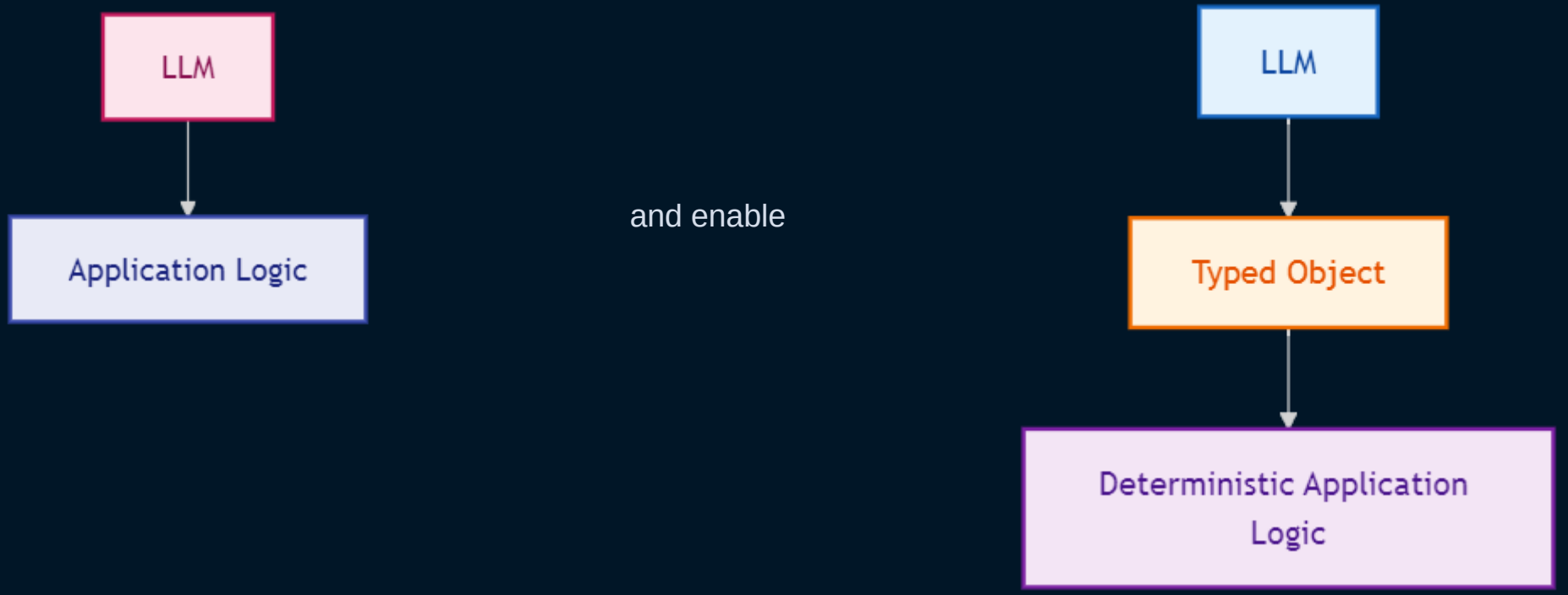
Structured Outputs

Use of Pydantic models for LLM outputs

with_structured_output() and function calling to generate the structured query

Architectural Benefit

Structured output reduces the amount of fragile string parsing required between:



LCEL (LangChain Expression Language) Design

- Use LangChain Expression Language to compose modular pipelines
- uses RunnableLambda, RunnablePassthrough, and mapped retrieval operations

LCEL Benefits

- composeability
- readable pipelines
- reusable components
- easy insertion of transformations
- clear data flow
- support for parallel/mapped execution

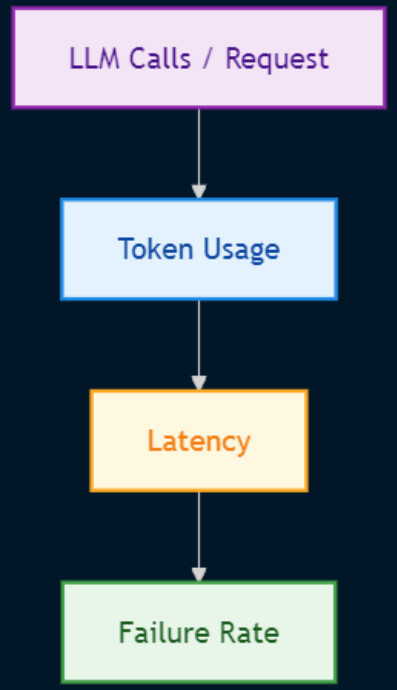
Performance and Cost

LLM Calls

The architecture may invoke an LLM for

- query routing
- structured query generation
- self-query metadata inference
- SQL generation
- SQL cleanup
- multi-query generation
- final answer generation

Therefore, a production implementation should carefully measure



RAG Fusion Cost

Generating five queries can increase:

- embedding/retrieval workload
- number of vector searches
- context candidates
- ranking computation
- Latency

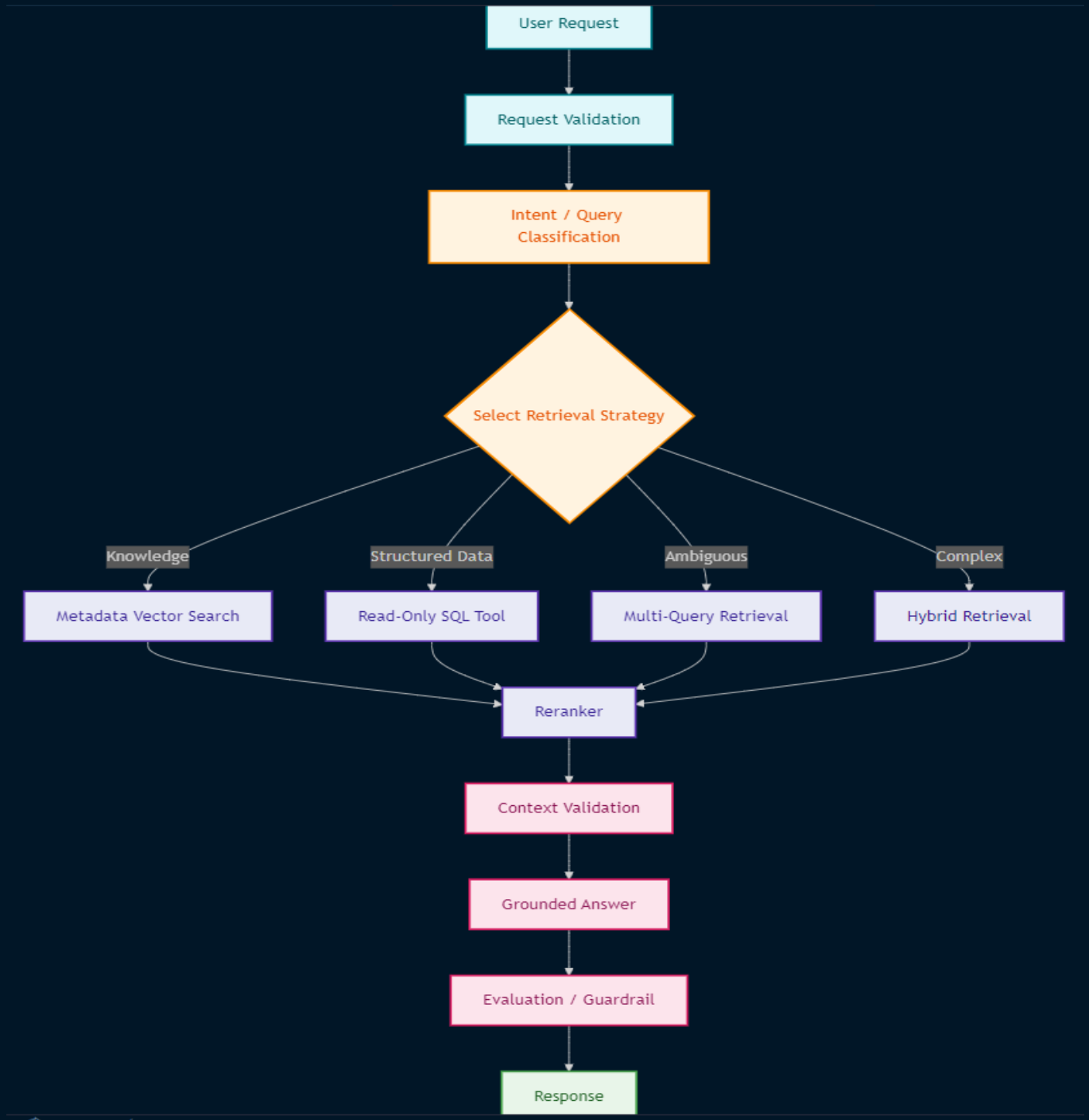
Optimization Opportunities

Recommended production techniques:

- cache query transformations
- classify simple queries before expensive routing
- dynamically choose the number of generated queries
- parallelize retrieval
- cap candidate documents
- use metadata filters early
- rerank only a limited candidate set
- cache embeddings
- track token budgets

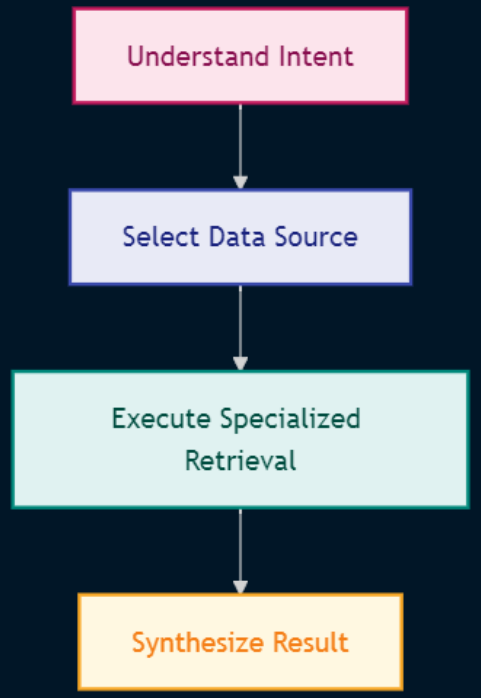
Production Architecture

A production evolution could transform the current chain-based routing system into an explicit stateful agentic retrieval workflow.

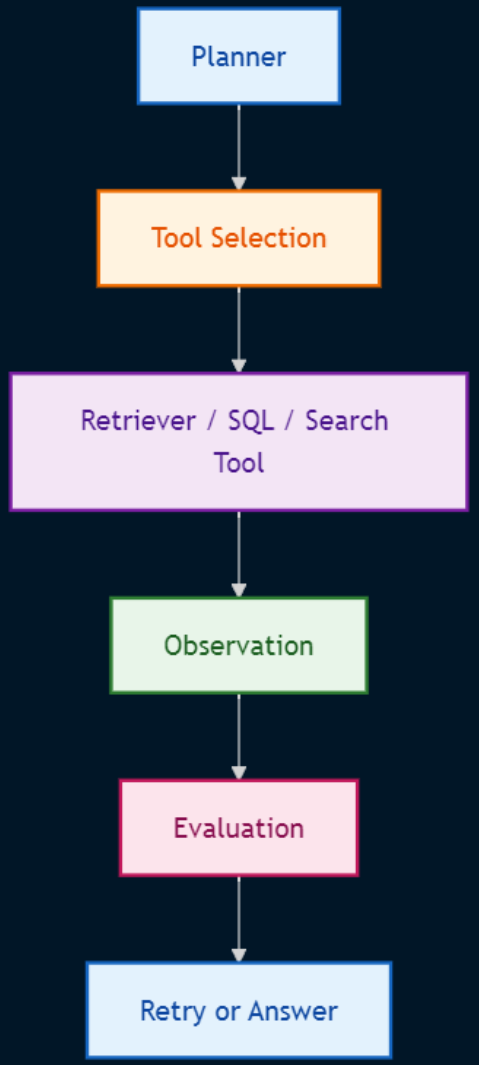


Agentic AI Evolution

The existing query router already demonstrates an important agentic design principle



A stronger Agentic architecture could extend this into



This would allow the system to decide dynamically whether it needs:

- vector retrieval
- SQL
- metadata filtering
- multi-query retrieval
- RAG Fusion
- additional evidence

Evaluation Strategy

A professional RAG evaluation suite should compare the retrieval strategies under the same test set.
Query Categories

Tourism Questions
Where are the best beaches in Cornwall?

Metadata Questions
What events are happening in Newquay?

Accommodation Questions
What offers are available in Cardiff?

Ambiguous Questions
Questions where routing between vector and SQL is non-trivial.

Multi-Intent Questions
Questions requiring several retrieval perspectives.

Recommended Metrics

Retrieval
- Recall@K
- Precision@K
- MRR
- nDCG
- Hit Rate

Generation
- answer relevance
- context relevance
- faithfulness
- citation correctness

System
- latency
- token consumption
- cost per request
- retrieval count
- SQL execution time
- routing accuracy